2026-10-16 · 第21节 · 学习任务管理系统 · 图文与代码练习

页面打开后，任务从哪里来？
让 Vue 读取后端和数据库

之前页面里的任务可以先写在 JavaScript 数组里，刷新后就回到原样。今天理解另一条链：Vue 发起请求，Spring Boot 验证登录身份，MyBatis 查询 MySQL，再把 JSON 任务列表交回页面。

> **今天的目标：**看懂固定数组为什么被替换为 `tasks = ref([])`，能沿 `onMounted → loadTaskData → loadTasks → apiRequest → GET /api/tasks` 追踪一次加载，并用刷新和重新登录证明数据来自数据库。

阅读顺序：先看第 1 ~ 3 节的地图，再看第 4 节真实代码，最后做第 5 ~ 7 节跟写和验证。功能已经提前写进项目，不等于今天已经掌握。

1. 先看整体：一次“打开页面”发生了什么

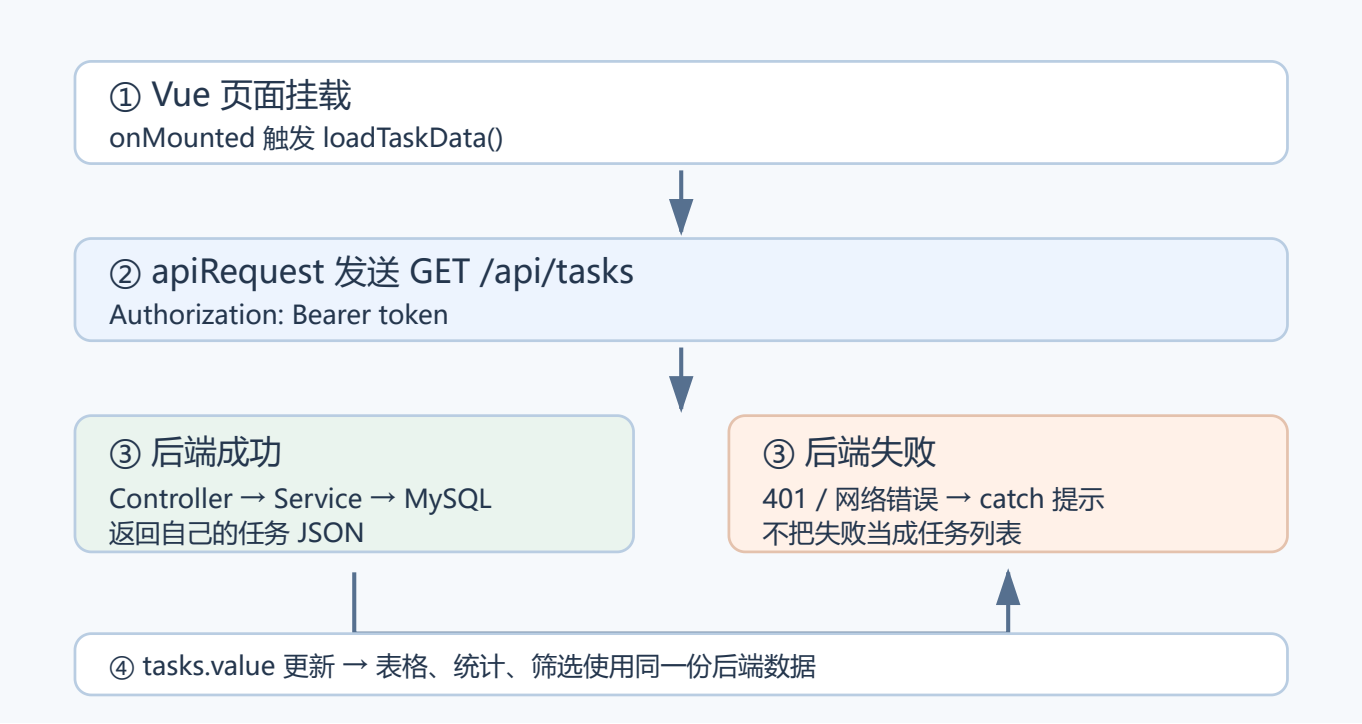


图 1：页面初次打开时，不是直接“显示数据库”，而是先发 HTTP 请求，再把响应放进 Vue 的响应式状态。

要记住三个角色：**页面状态**是 `tasks.value`；**接口**是 `GET /api/tasks`；**数据库**保存真正持久的数据。三者不是同一个东西。

2. 为什么删掉固定数组?

写法	页面打开时的数据来源	刷新后的结果
固定数组	直接使用 App.vue 里的示例对象	重新加载示例，新增内容容易丢失
后端加载	请求 API，API 查询当前用户的 MySQL 记录	重新请求数据库，已保存的任务仍在

所以“替换数组”不是把任务换一种写法，而是把数据源从**前端临时内存**换成**后端持久化数据**。前端仍然需要一个数组来渲染表格，只是这个数组由接口响应填充。

> 一个常见误解：浏览器 Network 里看到 `tasks 200`，只说明请求拿到了响应；要理解完整功能，还要知道后端通过 token 决定查哪个 user_id。

3. 先理解“响应式状态”

```
const tasks = ref([]);

tasks.value = await apiRequest("/api/tasks");
```

`ref([])` 创建一个可以被 Vue 追踪的状态。JavaScript 代码修改它时要写 `tasks.value`；模板中使用 `tasks`，Vue 会自动取出值并重新渲染表格。

这里的空数组不是“没有任务”，而是“请求完成前还没有放入任务”。请求完成后，响应 JSON 数组覆盖它；如果后端返回空数组，页面显示空状态，这也是真实结果。

加载和失败要分开

```
async function loadTasks() {
  tasksLoading.value = true;
  try {
    tasks.value = await apiRequest("/api/tasks");
  } finally {
    tasksLoading.value = false;
  }
}
```

加载状态控制表格的 loading 效果；请求错误由 `apiRequest` 和上层调用的 `catch` 处理。不要在请求失败时把 `tasks` 伪造为示例数据，否则用户会误以为数据库里还有这些任务。

4. 沿真实代码追踪一次 GET

4.1 App.vue：页面挂载

```
onMounted(async () => {
  if (!token.value) {
    router.push("/login");
    return;
  }
  await loadTaskData();
});
```

组件第一次进入页面时执行。先确认有没有 token，再加载任务和统计。不同版本的行号可能变化，以当前文件搜索到的函数为准。

4.2 loadTaskData 和 loadTasks

```
async function loadTasks() {
  tasksLoading.value = true;
  try {
    tasks.value = await apiRequest("/api/tasks");
  } finally {
    tasksLoading.value = false;
  }
}

async function loadTaskData() {
  await Promise.all([loadTasks(), loadStats()]);
}
```

`Promise.all` 表示任务列表和统计都开始请求；今天先重点看 `loadTasks`。`await` 等待响应完成，右侧赋值让模板得到新数据。

4.3 apiRequest：统一带上凭证

```
const response = await fetch(path, {
  ...options,
  headers: {
    Authorization: `Bearer ${token.value}`,
    "Content-Type": "application/json",
  },
});
return readJson(response);
```

请求地址是相对路径 `/api/tasks`；开发环境由 Vite 代理转到 8084。token 让后端知道当前用户。前端不把 `user_id` 放进请求让用户自由填写，这个归属由后端 JWT 决定。

4.4 后端接收并查询

```
@GetMapping
public List<TaskResponse> list(
    @RequestHeader(value = "Authorization", required = false)
    String authorization) {
  return service.list(authorization);
}

// Service 中的核心意思
Long userId = currentUserId(authorization);
return mapper.findByUserId(userId)
    .stream().map(TaskResponse::from).toList();
```

Controller 接住请求，Service 从已验证 token 得到 userId，Mapper 执行带 `WHERE user_id = #{userId}` 的查询，响应只包含当前用户的数据。

5. 今天要跟写的代码

先在草稿中写，不要一上来改正式文件。把下面函数拆成三句话：打开加载、请求列表、放入状态。

```
async function loadTasks() {
  tasksLoading.value = true;
  try {
    const data = await apiRequest("/api/tasks");
    tasks.value = data;
  } finally {
    tasksLoading.value = false;
  }
}
```

1. `tasksLoading.value = true`：开始请求，页面显示加载中。
2. `await apiRequest`：等待后端返回自己的任务数组。
3. `tasks.value = data`：把响应放进 Vue 状态，表格自动更新。
4. `finally`：无论成功失败都关闭 loading。

相近小练习

在草稿里写一个 `loadOnlyTasks`：调用同一个接口，成功后返回任务数组，失败时让错误继续抛出，不要吞掉错误。参考答案：

▶ 打开参考答案

这里特意没有加 `try/catch`：如果当前函数没有办法给用户一个合适提示，就让上层处理。捕获异常后什么都不做，会造成“页面看起来没反应”的问题。

6. 实操验证：证明数据来自后端

检查一：页面打开

保持后端 8084 和前端 5173 都运行，刷新任务页面。Network 中找到 `tasks`，预期状态 200；点开 Response，应是任务数组。不要只看表格，还要看这次请求。

检查二：刷新不丢数据

记住当前列表中的一个任务名称，刷新页面。若仍出现，是因为页面重新向后端读取，而不是浏览器保留了当前数组。

检查三：重新登录

退出后重新登录，打开任务页。应再次出现自己的任务；如果出现别人的任务，说明后端 user_id 隔离有问题，应停止继续操作。

检查四：无 token

清除会话或使用未登录页面访问任务接口，预期 401，页面跳转登录或提示重新登录。401 不是“列表为空”，而是身份没有通过验证。

> 今天不要求你新建任务；新增任务属于明天的“Vue 新建任务接入后端”。今天重点是看懂“已有数据怎样从后端进入页面”。

7. 复盘与完成标准

今天结束前，尝试不用看讲义回答：

- 为什么前端仍有 `tasks` 数组，但它不再是固定示例数组？
- 页面刷新后，哪个请求把数据库记录重新放进 `tasks.value`？
- 为什么前端不应该让用户提交 `user_id` 来决定归属？
- 请求失败时，为什么不能把任务列表替换成示例数据？

完成标准分三档记录：

层级	证据
读懂	能画出 onMounted → apiRequest → GET /api/tasks → tasks.value
跟写	能按参考写出 loadTasks，并说明 finally 的作用
会用	能独立把一个“加载固定数组”的小函数改成读取接口，并验证刷新仍保留数据

> **今天的定位：**后端任务查询已经存在，今天学习的是前端如何接入它；不要把“页面原本就能显示任务”当成今天已经掌握。